

Note on Data Encryption
Riccardo Lo Iacono

September 15, 2026

Contents.

1	Introduction to cryptography and codes	1
1.1	Some prerequisites	1

– 1 – Introduction to cryptography and codes.

The notion of cryptography is an old one; it derived from the necessity of hiding the content of a message when communicating through a medium (a channel for instance). In other words, cryptography was born as a way to make communications more secure. Similar to cryptography, stenography is another way to make communications secure, but the procedure behind it are completely different.

These notes will cover exclusively concepts of cryptography, and are meant to provide a broad view of the field. In addition, some code theory will be discussed half-way through these notes.

– 1.1 – Some prerequisites.

Although these notes are meant to give a broad view to the field, some mathematical background is somewhat required. For this reason, the present section will discuss some basic concepts of abstract algebra, as well as some concepts of discrete math.

Here, we introduce the notions of group, abelian group, ring and field; later, ??, we discuss some concepts of projective geometry since this provide the basic tools for modern cryptography.

– 1.1.1 – Groups, rings and fields.

To give the most accessible idea of what each of groups, rings and fields are, we can think of this as sets on which some operations are defined; in addition, such operations must satisfy some properties.

Formally speaking, given a set G and a binary operation $*$: $G \rightarrow G^1$, we say that $(G, *)$ forms a group if all of the following hold.

1. There exists an element $\mathbf{1}_G$ such that $a * \mathbf{1}_G = \mathbf{1}_G * a = a, \forall a \in G$. In other words, G must have a neutral element.
2. For all $a \in G$ there must exists a^{-1} such that $a * a^{-1} = a^{-1} * a = \mathbf{1}_G$. Meaning that each element must have an inverse.
3. The operation $*$ must be associative. That is, for all $a, b, c \in G$ it holds $a * (b * c) = (a * b) * c$.

Additionally, if $*$ is commutative, i.e., $a * b = b * a$, we say that $(G, *)$ is an abelian (or commutative) group.

¹Note that the use of $*$ for the binary operation should not fool the reader in to thinking it as the well known multiplication. Instead, it is a placeholder for any well defined operation.

Section 1 – Introduction to cryptography and codes

As for sets, groups may contain some smaller groups, to which we refer to as sub-groups. Formally, given $(G, *)$ a group, we say that $H \subseteq G$ forms a group under $*$ if $\mathbf{1}_G \in H$ and $(H, *)$ forms a group.

Beyond the set-theoretic relation, groups and subgroups are connected by a much more fundamental relation, that regarding their size. This result, known in the literature as *Lagrange's theorem for groups*, states that a finite group G , i.e. a group with finitely many elements, can only have subgroups H such that the number of elements in H divides the number of elements in G . We formally state Lagrange's theorem below.

Theorem (Lagrange's theorem for groups): *Let $(G, *)$ be a group, with $|G| < \infty$. Then, any subgroups $(H, *)$ of $(G, *)$ is such that*

$$|H| \mid |G|.$$

Although groups have some nice properties as the whole group-theory shows, more often than not we need something more powerful. To provide for this need, some more advanced structures were developed, the simplest of which is known as *ring*.

To understand rings, we can see them as an extension of groups by an additional operation. More precisely, we say that $R, +, *$ is a ring if

1. $(R, +)$ form an abelian group.
2. $*$ is associative. That is, for all $a, b, c \in R$ it holds $a * (b * c) = (a * b) * c$.
3. $*$ distributes over $+$. In other words, if for all $a, b, c \in R$ we have that $a * (b + c) = a * b + a * c$.

Rings can sometimes satisfy some additional properties, depending on which they take different names. These additional properties are the following.

1. There exists a neutral element for $*$.
2. $*$ is commutative.
3. Each element in R as inverse with respect to $*$.

A ring with all these additional properties is called *field*. In these notes we denote a generic field by either $\mathbb{F}$ or $\mathcal{K}$, while finite fields will be denoted by $\text{GF}(p)$, where p is usually a prime.

– 1.1.2 – Modular arithmetics.

Most of the groups and ring will be using in these notes, rise naturally from the notion of congruence modulo n , where n is an integer. These groups and rings will be denoted by $\mathbb{Z}_n$, or by $\mathbb{Z}_p$ if $n = p$ is prime.

Section 1 – Introduction to cryptography and codes

But what is congruence? Put in simple words, given two integers a, b , we say that a is congruent b modulo n , if we can write a as the sum of b and some multiple of n . That is,

$$a \equiv b \pmod{n} \iff \exists k \in \mathbb{Z} : a = b + kn.$$

Using the above definition, we get

$$\mathbb{Z}_n = \{m \in \mathbb{N} \mid m < n\}.$$

For instance, $\mathbb{Z}_{11} = \{0, 1, \dots, 9, 10\}$. Sometimes when dealing with rings we will write $\mathbb{Z}_n^*$ to mean the group/ring $\mathbb{Z}_n \setminus \{0\}$.

In most of the algorithms we will discuss, we often make use of the gcd between two given integers. Here we first recall what GCD is, and provide two algorithm to compute it.

Let a, b be two integers. We say that an integer d is the GCD of a, b if the following hold:

$$d \mid a \text{ and } d \mid b \tag{1}$$

and for any other integer d' such that Equation (1) holds, we have

$$d' \mid d.$$

In other words, the d is the GCD of a, b if any integer that divides a, b also divides d .

But how do we compute the GCD of two integers? A result in abstract algebra tells us that we can compute d as the difference between some multiple of a and some multiple of b . This result is known as *Bézout identity*. More precisely Bézout identity states that there exists two integers δ and ϵ , such that $d = \gcd(a, b) = a\delta + b\epsilon$. δ and ϵ are called Bézout coefficients.

In practise we compute the GCD by means of the Euclidean algorithm. Two variants exists, a simple one which returns just the GCD, and one which returns both the GCD and the Bézout coefficients. Here we present both, although for our needs the latter is preferred.

Algorithm 1 Euclidean algorithm
Input: Two integers a and b , with $a > b$.
Output: The greatest GCD of a and b .
Compute the sequence $\{r_i\}_{i \in \mathbb{N}}$ as follow: $r_0 \leftarrow a, r_1 \leftarrow b$
while $r_k \geq 1$ do
$r_{k+1} \leftarrow r_{k-1} - q_k r_k$
end
return the last computed r_k

Algorithm 2 Euclidean algorithm

Input: Two integers a and b , with $a > b$.

Output: The greatest GCD of a and b , and the Bézout coefficients.

Compute the sequences $\{r_i\}_{i \in \mathbb{N}}$, $\{s_i\}_{i \in \mathbb{N}}$, $\{t_i\}_{i \in \mathbb{N}}$ as follow:

$r_0 \leftarrow a, r_1 \leftarrow b$

$s_0 \leftarrow 1, s_1 \leftarrow 0$

$t_0 \leftarrow 0, t_1 \leftarrow 1$

while $r_k \geq 1$ **do**

$r_{k+1} \leftarrow r_{k-1} - q_i r_k$

$s_{k+1} \leftarrow s_{k-1} - q_i s_k$

$t_{k+1} \leftarrow t_{k-1} - q_i t_k$

end

return the last computed r_k as GCD and s_k, t_k as Bézout coefficients.